



9급 공무원 컴퓨터일반 요약노트 (마스터)

출제 분석: 2022-2026 9회차 180문제 기준 **2026 지방직(6/20) 대비 핵심 정리 빈출 순서:** 컴퓨터구조(24.4%) > 운영체제(15.6%) > 네트워크(13.9%) > 프로그래밍(10.6%) > DB(7.8%) > SW공학(7.2%) > 자료구조(6.7%) > 신기술(6.7%) > 알고리즘(3.9%) > 멀티미디어(2.2%) > 보안(0.6%)

제1장 컴퓨터 구조 (★★★★★ 최빈출)

1.1 수의 표현

진법 변환

- **2진수↔10진수:** 자릿값 가중치 합산 ($2^7 2^6 2^5 \dots 2^0$)
- **2진수↔16진수:** 4비트씩 묶어 변환 ($1111 = F, 1010 = A$)
- **10진수→2진수:** 2로 계속 나눠 나머지 역순

음수 표현 (★★★★★)

방식	표현 가능 범위 (n비트)	특징
부호-크기	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$	0 두 개(+0, -0)
1의 보수	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$	0 두 개, 비트 반전
2의 보수	$-2^{n-1} \sim +(2^{n-1}-1)$	0 한 개, 현 컴퓨터 표준

2의 보수 계산법

1. 절댓값 2진수 표현
2. 1의 보수(비트 반전) 계산
3. 1 더하기

예: 8비트로 -18 표현 -18 → 0001 0010 - 1의 보수 → 1110 1101 - +1 → **1110 1110** (= -18)

IEEE 754 부동소수점

- **단정도:** 32비트 = 부호(1) + 지수(8, bias=127) + 가수(23)
- **배정도:** 64비트 = 부호(1) + 지수(11, bias=1023) + 가수(52)
- 정규화: $1.xxx \times 2^n$ 형태로, 가수 앞 1은 생략

문자 인코딩

- **ASCII:** 7비트(128문자), 영문/숫자/특수기호
- **EBCDIC:** 8비트, IBM 메인프레임
- **유니코드(UTF-8):** 가변 길이(1-4바이트), 한글 3바이트
- **유니코드(UTF-16):** 2바이트 고정, 한글 2바이트

1.2 메모리 계층

계층	용량	속도	가격/비트	휘발성
레지스터	가장 작음	가장 빠름	가장 비쌈	휘발성
캐시 (SRAM)	작음	빠름	비쌈	휘발성
주기억 (DRAM)	중간	중간	중간	휘발성
보조기억 (HDD/SSD)	큼	느림	쌈	비휘발성

△ **함정**: DRAM과 SRAM 모두 휘발성이다. SRAM이 비휘발성이라고 하면 오답!

SRAM vs DRAM

구분	SRAM	DRAM
저장 소자	플립플롭	커패시터
리프레시	불필요	필요(주기적 충전)
속도	빠름	느림
용도	캐시	주기억장치
가격	비쌈	쌈

ROM·플래시

- **ROM**: Mask ROM, PROM, EPROM(자외선), EEPROM(전기), Flash ROM
- **SSD**: NAND Flash 기반, 비휘발성, 빠른 접근 시간

1.3 캐시 메모리 (★★★★)

지역성 원리

- **시간 지역성(Temporal)**: 한번 참조된 데이터는 곧 다시 참조 (예: 반복문)
- **공간 지역성(Spatial)**: 인접한 데이터가 함께 참조 (예: 배열)
- **순차 지역성(Sequential)**: 순차적 명령어 실행

사상(Mapping) 방식

방식	특징	장단점
직접 사상	한 주기억 블록 → 정해진 한 캐시 블록	단순, but 충돌 잦음
연관 사상	한 주기억 블록 → 어느 캐시든 OK	충돌 적음, but 검색 비용
집합 연관	n-way로 묶어 직접+연관 혼합	절충안 (현대 CPU 주류)

캐시 적중률 (Hit Ratio)

- **평균 접근 시간** = $h \times T_c + (1-h) \times T_m$
(h=적중률, Tc=캐시시간, Tm=주기억시간, 미스 시 캐시도 거쳐감 모델은 + Tm)
- **예**: Tc=5ns, Tm=50ns, 평균=10ns → $h = 0.9$

캐시 쓰기 정책

- **Write-Through**: 캐시·주기억 동시 쓰기 (일관성 유지)
- **Write-Back**: 캐시만 쓰고 dirty 표시, 나중에 일괄 (성능 ↑)

1.4 CPU 구조

CPU 구성요소

- **ALU**(연산장치): 산술/논리 연산
- **CU**(제어장치): 명령 해석, 신호 생성
- **레지스터**: PC(다음 명령어 주소), IR(현재 명령어), MAR/MBR, ACC, SR
- **버스**: 데이터 버스, 주소 버스, 제어 버스

명령어 처리 사이클 (★★★★)

인출(Fetch) → 해석(Decode) → 실행(Execute) → 저장(Write back) → 인터럽트 처리

파이프라인

- **단계:** IF(Fetch) → ID(Decode) → EX(Execute) → MEM(Memory) → WB(Writeback)
- **해저드:** 자료 해저드(RAW), 제어 해저드(분기), 구조 해저드(자원 충돌)
- **속도 향상:** 이상적으로 단계 수만큼 향상, 실제로는 해저드로 제한

RISC vs CISC

구분	RISC	CISC
명령어 수	적음(단순)	많음(복잡)
명령어 길이	고정	가변
실행 시간	1 사이클/명령어	다양
파이프라이닝	용이	어려움
대표	ARM, MIPS, RISC-V	x86, x64

어드레싱 모드

- **즉치(Immediate):** 명령어에 데이터 직접 포함
- **직접(Direct):** 주소 필드가 메모리 주소
- **간접(Indirect):** 주소 필드가 가리키는 주소가 또 주소
- **레지스터:** 주소 필드가 레지스터 번호
- **인덱스(Index):** 베이스 + 인덱스 레지스터

인터럽트

- **외부:** 입출력, 타이머, 전원 이상
- **내부:** 0 나눗셈, 오버플로 (트랩, 예외)
- **소프트웨어:** SVC(시스템 콜)
- **처리 순서:** PC 저장 → ISR 실행 → 복귀

1.5 디지털 논리

기본 게이트

- AND, OR, NOT, NAND, NOR, XOR, XNOR
- 진리표 4가지 입력(00,01,10,11)에 대한 출력

부울 대수 법칙

- **드모르간:** $(A \cdot B)' = A' + B'$, $(A + B)' = A' \cdot B'$
- **흡수:** $A + A \cdot B = A$, $A \cdot (A + B) = A$
- **분배:** $A \cdot (B + C) = A \cdot B + A \cdot C$
- **상보:** $A + A' = 1$, $A \cdot A' = 0$

조합 회로

- **반가산기(HA):** $S = A \oplus B$, $C = A \cdot B$
- **전가산기(FA):** $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + (A \oplus B)C_{in}$
- **디코더:** $n \rightarrow 2^n$, **인코더:** $2^n \rightarrow n$, **멀티플렉서:** $2^n \rightarrow 1$

순차 회로 (플립플롭)

- **RS:** Set/Reset (S=R=1 금지)
- **JK:** J=K=1 시 토글
- **D:** 입력 그대로 저장 (지연)
- **T:** T=1 시 토글

카운터·시프트 레지스터

- **카운터**: 2ⁿ 상태, 모듈로-n 가능
- **시프트**: 좌/우 시프트로 곱/나눗셈, 산술/논리 구분

1.6 RAID

레벨	방식	특징
RAID 0	스트라이핑	성능 ↑, 패리티 없음, 장애 시 데이터 전손
RAID 1	미러링	가용성 ↑, 용량 50% 낭비
RAID 5	분산 패리티	패리티 분산, 1개 디스크 장애 복구
RAID 6	이중 패리티	2개 디스크 동시 장애 복구, 쓰기 느림
RAID 10	1+0	미러링 후 스트라이핑, 비용 ↑ 성능·가용성 ↑

⚠ **함정**: RAID 6은 RAID 5보다 **패리티 더 많고 더 느리다** (반대 X)

제2장 운영체제 (★★★★★)

2.1 OS 기본 개념

OS 역할

- 자원 관리(CPU, 메모리, 입출력)
- 프로세스 관리
- 사용자 인터페이스
- 보안/보호

OS 구조

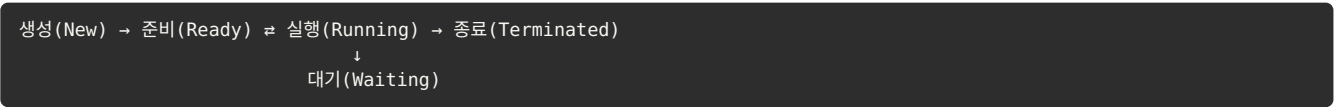
- **모놀리식**: 하나의 큰 커널 (Unix 초기)
- **계층화**: 계층별 분리 (THE OS)
- **마이크로커널**: 최소 커널 + 사용자 공간 서비스 (Mach, MINIX)
- **모듈**: 동적 로딩 모듈 (Linux)

시스템 콜

- fork(): 자식 프로세스 복제
- exec(): 새 프로그램 실행
- wait(): 자식 종료 대기
- pipe(), open(), read(), write()

2.2 프로세스·스레드

프로세스 상태



PCB(프로세스 제어 블록) 구성

- 프로세스 ID(PID), 상태
- PC, 레지스터 값
- 메모리 정보(베이스, 한계)
- 스케줄링 정보(우선순위, 큐 포인터)
- I/O 정보, 회계 정보

프로세스 vs 스레드

구분	프로세스	스레드
자원	독립 메모리 공간	동일 프로세스 내 공유
통신	IPC (파이프, 메시지 큐)	메모리 직접 공유
컨텍스트 스위칭	비용 큼	비용 작음
보호	강함	약함

IPC (Inter-Process Communication)

- 공유 메모리
- 파이프(익명·명명)
- 메시지 큐
- 시그널
- 소켓

2.3 CPU 스케줄링 (★★★★★)

비선점형 (Non-preemptive)

알고리즘	동작	특징
FCFS	도착 순서	호위 효과 가능
SJF	가장 짧은 작업 우선	평균 대기시간 최소(이론)
HRN	응답률 최고 우선	응답률 = (대기+실행)/실행
우선순위	우선순위 높은 것	기아 가능

선점형 (Preemptive)

알고리즘	동작	특징
SRT	SJF의 선점 버전	짧은 작업 도착 시 선점
RR	타임 슬라이스 순환	시분할 시스템 표준
MLFQ	다단계 피드백 큐	UNIX 변형

평균 대기시간 계산법

- 도착 시간 = 0으로 가정 시: 대기 = 시작 시간
- 예) SJF: P1(10), P2(4), P3(6) → P2→P3→P1 순
 - P2 대기 0, P3 대기 4, P1 대기 10 → 평균 (0+4+10)/3 = 4.67ms

△ **합정**: FCFS는 비선점 / RR은 선점 / **HRN은 비선점, SRT는 선점**

2.4 동기화

임계구역 문제

- **상호 배제**: 한 번에 한 프로세스만 진입
- **진행**: 임계구역 밖 프로세스가 막지 못함
- **한정된 대기**: 무한히 기다리지 않음

동기화 도구

- **세마포어(Semaphore)**: P(wait), V(signal) 연산, 정수형
 - 이진 세마포어: 0/1 (뮤텍스와 유사)
 - 카운팅 세마포어: 0~N
- **뮤텍스(Mutex)**: 소유권 개념, lock/unlock
- **모니터**: 고수준, 자동 상호 배제

교착상태 (Deadlock) 4가지 필요조건

- 1. 상호 배제 (Mutual Exclusion)
- 2. 점유와 대기 (Hold and Wait)
- 3. 비선점 (No Preemption)
- 4. 순환 대기 (Circular Wait)

△ 4가지 모두 만족해야 발생. 하나라도 깨면 예방 가능.

교착상태 해결

- 예방: 4조건 중 하나 깨기 (자원 한꺼번에 할당 등)
- 회피: 은행원 알고리즘 (Safe state 유지)
- 탐지·복구: 대기 그래프 사이클 검출 후 강제 종료
- 무시: 거의 발생 안 하면 무시 (Ostrich 알고리즘)

2.5 메모리 관리

메모리 할당 (가변 분할)

- 최초 적합(First Fit): 충분한 첫 번째 빈 공간
- 최적 적합(Best Fit): 가장 작게 남는 곳 (검색 비용 ↑, 작은 단편화 ↑)
- 최악 적합(Worst Fit): 가장 큰 빈 공간 (큰 작업 불가)

단편화

- 내부 단편화: 할당된 공간 내 미사용 (고정 분할)
- 외부 단편화: 빈 공간들이 흩어짐 (가변 분할) → 압축으로 해결

페이징 (Paging)

- 페이지(고정 크기) 단위로 메모리 분할
- 페이지 테이블로 가상↔물리 주소 변환
- TLB: 페이지 테이블 캐시
- 외부 단편화 해소, 단 내부 단편화 발생 가능

세그멘테이션

- 논리적 단위(코드, 데이터, 스택)로 분할
- 가변 크기, 외부 단편화 발생
- 보호·공유 용이

가상 메모리 + 페이지 교체

알고리즘	동작	특징
FIFO	가장 먼저 들어온 페이지 교체	Belady 이상 현상
LRU	가장 오래 사용 안 됨	시간 정보 필요
LFU	사용 빈도 최저	초기 페이지 불리
Optimal	미래 가장 안 쓸 페이지	이론 최적 (구현 불가)
2nd Chance	FIFO + 참조 비트	클럭 알고리즘

Belady 이상 현상

- FIFO에서 프레임 수를 늘렸는데 페이지 폴트가 증가하는 현상
- LRU, Optimal 등 스택 알고리즘은 발생하지 않음

2.6 디스크 스케줄링

알고리즘	동작	특징
FCFS	요청 순서	효율 낮음
SSTF	가장 가까운 트랙 우선	기아 가능
SCAN	한쪽 끝까지 → 반대로 (엘리베이터)	균형, 끝 가까운 거 유리
C-SCAN	한 방향만 (끝나면 점프)	균등한 대기
LOOK	SCAN인데 끝까지 안 감	더 효율적
C-LOOK	C-SCAN의 LOOK 버전	가장 균형

제3장 네트워크 (★★★★★)

3.1 OSI 7 계층 (★★★★★)

계층	이름	PDU	장비	프로토콜
7	응용	Data	게이트웨이	HTTP, FTP, SMTP, DNS, Telnet
6	표현	Data	-	SSL/TLS, JPEG, MPEG
5	세션	Data	-	NetBIOS, RPC
4	전송	Segment	-	TCP, UDP, SCTP
3	네트워크	Packet	라우터	IP, ICMP, ARP, IGMP
2	데이터 링크	Frame	스위치, 브리지	Ethernet, PPP, HDLC
1	물리	Bit	리피터, 허브	RS-232, 케이블

⚠ **함정:** ARP/ICMP는 **네트워크 계층** (응용 아님!)

TCP/IP 4 계층

1. 네트워크 인터페이스 (= OSI 1+2)
2. 인터넷 (= OSI 3)
3. 전송 (= OSI 4)
4. 응용 (= OSI 5+6+7)

3.2 TCP vs UDP (★★★★★)

구분	TCP	UDP
연결	연결 지향 (3-way handshake)	비연결
신뢰성	보장 (재전송, ACK)	보장 안 함
순서	보장	보장 안 함
흐름·혼잡 제어	있음	없음
헤더 크기	20-60바이트	8바이트 (고정)
속도	느림	빠름
용도	HTTP, FTP, SMTP	DNS, DHCP, SNMP, VoIP, 게임

TCP 3-way handshake

```

클라이언트 → SYN → 서버
클라이언트 ← SYN+ACK ← 서버
클라이언트 → ACK → 서버 (연결 수립)

```

TCP 혼잡 제어

- **슬로 스타트:** cwnd 지수 증가 (1→2→4→8...)

- **혼잡 회피**: ssthresh 도달 시 선형 증가 (+1)
- **빠른 재전송**: 3중 중복 ACK 수신 시 재전송
- **빠른 회복**: cwnd를 절반으로 (Reno)
- ⚠ **Timeout** 시: ssthresh=cwnd/2, cwnd=1로 리셋

3.3 IP 주소

IPv4 클래스

클래스	첫 비트	범위	네트워크/호스트
A	0xxx	0.0.0.0 ~ 127.255.255.255	8/24비트
B	10xx	128.0.0.0 ~ 191.255.255.255	16/16비트
C	110x	192.0.0.0 ~ 223.255.255.255	24/8비트
D	1110	224.0.0.0 ~ 239.255.255.255	멀티캐스트
E	1111	240.0.0.0 ~ 255.255.255.255	예약

서브네팅

- **/24를 4개로 나누기**: 2비트 필요 → /26
- **/26 마스크**: 255.255.255.192
- **호스트 수**: $2^n - 2$ (네트워크·브로드캐스트 주소 제외)
- **예**: /26 → $2^6 - 2 = 62$ 개

CIDR 표기 핵심값

CIDR	마스크	호스트 수
/24	255.255.255.0	254
/25	255.255.255.128	126
/26	255.255.255.192	62
/27	255.255.255.224	30
/28	255.255.255.240	14
/29	255.255.255.248	6
/30	255.255.255.252	2

IPv6

- 128비트 (16바이트), 16진수 8그룹 콜론 표기
- 헤더 40바이트 고정 (IPv4보다 큼)
- 보안(IPSec) 기본 지원
- 브로드캐스트 없음, 멀티캐스트로 대체
- 자동 주소 설정 (SLAAC)

3.4 주요 프로토콜·포트

프로토콜	포트	TCP/UDP	용도
FTP	20/21	TCP	파일 전송
SSH	22	TCP	보안 셸
Telnet	23	TCP	원격 접속
SMTP	25	TCP	메일 송신
DNS	53	UDP(+TCP)	도메인 변환
DHCP	67/68	UDP	IP 자동 할당
HTTP	80	TCP	웹
POP3	110	TCP	메일 수신
IMAP	143	TCP	메일 수신
HTTPS	443	TCP	보안 웹
SMB	445	TCP	파일 공유
SNMP	161/162	UDP	네트워크 관리

⚠ **합정: DNS = 53, DHCP = 67/68** (혼동 주의)

ICMP

- 네트워크 계층 프로토콜
- ping, traceroute에 사용
- 오류 보고·진단

라우팅

- **정적 라우팅**: 수동 설정
- **동적 라우팅**: 자동 갱신
 - **거리 벡터**: RIP (홉 수, 최대 15)
 - **링크 상태**: OSPF (다익스트라, 대규모망)
 - **경로 벡터**: BGP (AS 간 라우팅)

제4장 프로그래밍 (★★★★)

4.1 C 언어 핵심

자료형 크기

자료형	크기	범위 (signed)
char	1바이트	-128 ~ 127
short	2바이트	-32,768 ~ 32,767
int	4바이트	$-2^{10247} \sim +2^{10247}$
long	4 또는 8바이트	환경 의존
long long	8바이트	$-2^{63} \sim 2^{63}-1$
float	4바이트	$\pm 3.4 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$
double	8바이트	$\pm 1.7 \times 10^{-308} \sim \pm 1.7 \times 10^{308}$

연산자 우선순위

1. () [] -> .
2. ! ~ ++ -- * & (단항)
3. ◦ / %
4. ◦

■

5. <<>>
6. < <= > >=
7. == !=
8. & ^ |
9. && ||
10. ?: (삼항)
11. = += -= 등 (대입)
12. ,

포인터·배열

- `*p` = p가 가리키는 곳의 값
- `&x` = x의 주소
- `arr[i]` $\equiv$ `*(arr + i)`
- `arr` $\equiv$ `&arr[0]` (배열명은 첫 요소 주소)
- 포인터 산술: `p + 1` 은 자료형 크기만큼 증가

함수 호출

- **값에 의한 호출(Call by Value)**: 복사 전달, swap 불가
- **참조에 의한 호출(Call by Reference)**: 포인터/참조 전달, swap 가능
- C는 값 호출만 지원, 포인터로 참조 흉내

비트 연산

- `&` AND, `|` OR, `^` XOR, `~` NOT
- `<<` 좌 시프트 ($\times 2$), `>>` 우 시프트 ($\div 2$)
- 부호 있는 우 시프트: 산술(부호 비트 유지)
- 부호 없는 우 시프트: 논리(0 채움)

함정 코드 패턴

```
int a = 5;
printf("%d %d %d", a++, ++a, a); // 컴파일러마다 다름 (UB)

int *p = &a;
*p++; // ++ 우선순위 높음. *p한 후 p 증가
(*p)++; // p가 가리키는 값을 증가
```

4.2 Java 핵심

기본

- **객체 지향**: 클래스, 상속, 다형성, 캡슐화
- **JVM**: 바이트코드 실행, 플랫폼 독립적
- **main** 메서드: `public static void main(String[] args)`
- 가비지 컬렉션: 자동 메모리 관리

static

- 클래스 변수/메서드: 모든 인스턴스 공유
- static 블록: 클래스 로딩 시 1번 실행
- 인스턴스 없이 호출 가능

접근 제어자

제어자	같은 클래스	같은 패키지	자식 클래스	외부
private	O	X	X	X
default	O	O	X	X
protected	O	O	O	X
public	O	O	O	O

상속·다형성

```
class Parent { void m() {...} }
class Child extends Parent { void n() {...} } // 오버라이딩
Parent p = new Child(); // 업캐스팅, 다형성
p.m(); // Child의 m() 실행 (동적 바인딩)
```

4.3 Python 핵심

자료형

- **숫자**: int (가변 길이), float
- **시퀀스**: list[], tuple(), str
- **매핑**: dict{}
- **집합**: set{}, frozenset

리스트 함수

- **append(x)**: 끝에 추가
- **extend(iterable)**: 시퀀스 확장
- **insert(i, x)**: 위치 삽입
- **pop(i)**: 제거+반환
- **+**: 새 리스트 생성 (in-place 아님)
- **+=**: extend와 동일 (in-place)

⚠ **함정**: `y = x` 는 **참조 복사**, 같은 객체 공유. 깊은 복사는 `copy.deepcopy()`.

컴프리헨션

```
[x**2 for x in range(10) if x % 2 == 0] # 리스트
{x: x**2 for x in range(5)} # 딕셔너리
{x for x in lst} # 집합
```

람다·고차함수

```
lambda x: x + 1
map(func, iterable) # 각 원소에 적용
filter(func, iterable) # 조건 만족만
reduce(func, iterable) # 누적 (functools)
```

예외 처리

```
try:
    위험한 코드()
except ValueError as e:
    처리()
except Exception:
    기본_처리()
else:
    정상_시()
finally:
    항상_실행()
```

제5장 데이터베이스 (★★★)

5.1 데이터 모델

관계 모델 용어

- **릴레이션(테이블)**: 행과 열로 구성
- **튜플(행)**: 하나의 레코드
- **속성(열)**: 컬럼
- **도메인**: 속성이 가질 수 있는 값의 집합
- **차수(Degree)**: 속성 수
- **카디널리티**: 튜플 수

키

- **슈퍼키**: 튜플 식별 가능한 모든 키 (임여 포함)
- **후보키**: 최소 슈퍼키
- **기본키**: 후보키 중 선택된 1개 (NOT NULL, UNIQUE)
- **대체키**: 후보키 - 기본키
- **외래키**: 다른 테이블의 기본키 참조

무결성 제약

- **개체(Entity) 무결성**: 기본키 NULL 불가
- **참조(Referential) 무결성**: 외래키는 참조 테이블의 기본키이거나 NULL
- **도메인(Domain) 무결성**: 속성 값은 도메인 내

5.2 정규화 (★★★)

정규형	조건	제거
1NF	원자 값만	다치/그룹 속성
2NF	1NF + 완전 함수 종속	부분 함수 종속
3NF	2NF + 비주요속성 → 비주요속성 X	이행 함수 종속
BCNF	모든 결정자가 슈퍼키	(강한 3NF)
4NF	BCNF + 다치 종속 X	다치 종속
5NF	4NF + 조인 종속 X	조인 종속

함수 종속 (FD)

- $A \rightarrow B$: A가 B를 결정 (A 같으면 B 같음)
- **부분 함수 종속**: 복합키의 일부에만 종속 (2NF 위반)
- **이행 함수 종속**: $A \rightarrow B, B \rightarrow C \implies A \rightarrow C$ (3NF 위반)

폐포(Closure) 계산

- X^+ : X로부터 추론 가능한 모든 속성 집합
- 모든 속성을 포함하면 X는 후보키 후보

5.3 SQL

DDL (정의)

```
CREATE TABLE 학생 (  
    학번 INT PRIMARY KEY,  
    이름 VARCHAR(20) NOT NULL,  
    전공 VARCHAR(30)  
);  
ALTER TABLE 학생 ADD COLUMN 학년 INT;  
DROP TABLE 학생;
```

DML (조작)

```
SELECT * FROM 학생 WHERE 전공 = '컴퓨터';
INSERT INTO 학생 VALUES (1, '홍길동', '컴퓨터');
UPDATE 학생 SET 학년 = 3 WHERE 학번 = 1;
DELETE FROM 학생 WHERE 학번 = 1;
```

실행 순서 (★★★★)

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

JOIN

- **INNER JOIN**: 양쪽 매칭만
- **LEFT OUTER**: 왼쪽 전부 + 오른쪽 매칭 (NULL 채움)
- **RIGHT OUTER**: 오른쪽 전부 + 왼쪽 매칭
- **FULL OUTER**: 양쪽 전부
- **CROSS JOIN**: 카티전 곱 ($m \times n$)

집계 함수

- COUNT, SUM, AVG, MIN, MAX
- GROUP BY와 함께 사용
- WHERE는 행 필터, HAVING은 그룹 필터

5.4 트랜잭션

ACID

- **원자성(Atomicity)**: 전부 실행 or 전혀 실행 안 됨
- **일관성(Consistency)**: 무결성 제약 유지
- **고립성(Isolation)**: 동시 트랜잭션 간 격리
- **지속성(Durability)**: 커밋된 결과는 영구 보존

고립 수준 (Isolation Level)

수준	Dirty Read	Non-repeatable	Phantom
READ UNCOMMITTED	O	O	O
READ COMMITTED	X	O	O
REPEATABLE READ	X	X	O
SERIALIZABLE	X	X	X

회복

- **로그 기반**: 트랜잭션 로그 → REDO/UNDO
- **검사점(Checkpoint)**: 회복 범위 축소
- **그림자 페이징**: 페이지 단위 복원

NoSQL vs RDBMS

구분	RDBMS	NoSQL
스키마	고정	유연
트랜잭션	ACID	BASE (결과적 일관성)
확장	수직	수평
종류	MySQL, Oracle	MongoDB(문서), Redis(키-값), Cassandra(컬럼), Neo4j(그래프)

제6장 소프트웨어 공학 (★★★★)

6.1 개발 방법론

폭포수 (Waterfall)

- 순차적 단계 진행: 요구분석 → 설계 → 구현 → 테스트 → 유지보수
- 단계 완료 후 다음 진행, 변경 어려움

프로토타입

- 시제품 제작 후 피드백 반복
- 요구사항 명확화에 도움

나선형 (Spiral)

- 위험 분석 중심 반복
- 4단계: 계획 → 위험 분석 → 개발 → 평가

애자일 (Agile) ★★★

- 선언문 4가지:
 1. 프로세스보다 **개인과 상호작용**
 2. 포괄적 문서보다 **작동하는 SW**
 3. 계약 협상보다 **고객과의 협력**
 4. 계획 따르기보다 **변화에 대응**
- 주요 기법: 스크럼(Scrum), XP, 칸반(Kanban), Lean
- 짧은 반복(스프린트), 점진적 개발

6.2 결합도·응집도 (★★★★)

결합도 (낮을수록 좋음)

강도	결합도	의미
약함 ↑	자료(Data)	매개변수로 단순 데이터만
	스탬프(Stamp)	구조체·배열 일부만 사용
	제어(Control)	제어 플래그 전달
	외부(External)	외부 자원 공유
	공통(Common)	전역 변수 공유
강함 ↓	내용(Content)	다른 모듈 내부 직접 참조

응집도 (높을수록 좋음)

강도	응집도	의미
약함 ↓	우연적	무관한 기능 묶음
	논리적	유사 기능 묶음
	시간적	같은 시점 실행
	절차적	순서대로 실행
	통신적	같은 데이터 사용
	순차적	출력이 다음 입력
강함 ↑	기능적	단일 목적

6.3 테스트

화이트박스 (코드 구조 기반)

- 기본 경로 테스트(McCabe)
- 문장/분기/조건/경로 커버리지

- 데이터 흐름, 루프 테스트
- 순환 복잡도 = $E - N + 2$ (E =간선, N =노드)

블랙박스 (명세 기반)

- 동등 분할 (Equivalence Partitioning)
- 경계값 분석 (Boundary Value Analysis)
- 원인-결과 그래프
- 비교 테스트, 오류 추측

테스트 단계

- 단위(Unit) → 통합(Integration) → 시스템(System) → 인수(Acceptance)
- V-모델: 개발 단계별 대응 테스트

6.4 UML 9종 (★★★)

구조 다이어그램 (4종)

- 클래스(Class): 클래스와 관계
- 객체(Object): 인스턴스 표현
- 컴포넌트: 물리적 컴포넌트
- 배치(Deployment): 하드웨어 배치

행위 다이어그램 (5종)

- 유스케이스(Use case): 기능 요구사항
- 순차(Sequence): 시간 순서대로 메시지
- 협력(Collaboration/Communication): 객체 간 협력
- 상태(State): 상태 전이
- 활동(Activity): 작업 흐름

6.5 디자인 패턴 (GoF 23종)

생성 패턴 (5)

- 싱글톤: 인스턴스 1개
- 팩토리 메소드: 객체 생성 위임
- 추상 팩토리: 관련 객체 군 생성
- 빌더: 단계적 객체 생성
- 프로토타입: 복제로 생성

구조 패턴 (7)

- 어댑터(Adapter): 인터페이스 변환
- 브리지: 추상/구현 분리
- 컴포지트: 트리 구조
- 데코레이터: 기능 동적 추가
- 퍼사드(Facade): 복잡한 시스템 단순화
- 플라이웨이트: 공유로 메모리 절약
- 프록시: 대리 객체

행위 패턴 (11)

- 옵저버(Observer): 1:N 통지
- 전략(Strategy): 알고리즘 교체
- 템플릿 메소드: 골격 정의
- 이터레이터: 순회 통일
- 커맨드: 요청 캡슐화
- 책임 연쇄(Chain of Responsibility): 처리자 체인
- 메멘토: 상태 저장/복원
- 상태(State): 상태별 동작

- 방문자(Visitor): 연산 분리
- 중재자(Mediator): 객체 간 통신 중개
- 인터프리터: 문법 해석

6.6 품질·관리 모델

CMMI 단계

- 초기 (Initial): 임시적
- 관리 (Managed): 프로젝트 단위
- 정의 (Defined): 표준화
- 정량적 관리 (Quantitatively Managed)
- 최적화 (Optimizing)

SOLID 원칙 (객체지향 5원칙)

- SRP: 단일 책임
- OCp: 개방-폐쇄 (확장 O, 변경 X)
- LSP: 리스코프 치환
- ISP: 인터페이스 분리
- DIP: 의존성 역전

Lehman 법칙

- 지속적 변경, 복잡도 증가, 자기 규제, 조직 안정성 등

제7장 자료구조 (★★★)

7.1 선형 자료구조

배열·연결 리스트

구분	배열	연결 리스트
접근	O(1) 임의 접근	O(n) 순차 접근
삽입/삭제	O(n)	O(1) (위치 알면)
메모리	연속	비연속 (포인터)

스택 (LIFO)

- 연산: push, pop, peek/top
- 응용: 함수 호출, 후위 표기식, 괄호 매칭, DFS, 진법 변환

큐 (FIFO)

- 연산: enqueue, dequeue, front, rear
- 종류: 일반 큐, 원형 큐, 덱(Deque), 우선순위 큐
- 응용: 프로세스 스케줄링, BFS, 프린터 큐

7.2 트리

용어

- 루트, 부모/자식, 형제, 잎(리프)
- 레벨: 루트=0
- 높이: 루트~잎의 최장 경로
- 차수: 자식 수의 최대값

이진 트리 종류

- 완전 이진 트리(Complete): 마지막 레벨 왼쪽 채워짐

- **포화 이진 트리(Full/Perfect)**: 모든 레벨 꽉 참
- **편향 이진 트리(Skewed)**: 한쪽 자식만
- **이진 탐색 트리(BST)**: 왼쪽 < 루트 < 오른쪽

순회 (Traversal) ★★★

순서	동작
전위 (Pre-order)	루트 → 왼쪽 → 오른쪽
중위 (In-order)	왼쪽 → 루트 → 오른쪽 (BST는 정렬)
후위 (Post-order)	왼쪽 → 오른쪽 → 루트
레벨 순서 (Level)	BFS, 큐 사용

힙 (Heap)

- **최대 힙**: 부모 ≥ 자식 (루트가 최대)
- **최소 힙**: 부모 ≤ 자식 (루트가 최소)
- 완전 이진 트리, 우선순위 큐 구현
- 삽입·삭제: $O(\log n)$

이진 탐색 트리 (BST)

- 검색·삽입·삭제: 평균 $O(\log n)$, 최악 $O(n)$
- 균형 BST: AVL, Red-Black Tree → 보장 $O(\log n)$

7.3 해시 (Hash)

해시 함수

- **나눗셈법**: $h(k) = k \bmod m$
- **곱셈법**: $h(k) = \lfloor m \times (k \times A - \lfloor k \times A \rfloor) \rfloor$
- **폴딩법**: 비트 단위 분할 후 합산

충돌 해결

방법	동작
선형 조사(Linear Probing)	$(h+i) \bmod m$ 으로 다음 빈 칸
이차 조사(Quadratic)	$(h+i^2) \bmod m$
이중 해싱	다른 해시 함수로 간격 결정
체이닝(Chaining)	연결 리스트로 충돌 키 저장

해시 성능

- 적재율(load factor) $\alpha = n/m$
- 평균 검색 시간 $O(1)$ (좋은 해시 함수)

7.4 그래프

표현

- **인접 행렬**: $V \times V$, $O(V^2)$ 공간, 간선 확인 $O(1)$
- **인접 리스트**: $O(V+E)$ 공간, 간선 확인 $O(\text{degree})$

탐색

- **DFS**: 스택(or 재귀), 깊이 우선, 시간 $O(V+E)$
- **BFS**: 큐, 너비 우선(레벨 순), 최단 경로(가중치 없음)

최단 경로

- **다익스트라**: 음수 가중치 없음, $O((V+E) \log V)$

- 벨만-포드: 음수 가중치 OK, O(VE)
- 플로이드-워셜: 모든 쌍 최단 경로, O(V³)

최소 신장 트리 (MST)

- 크루스칼(Kruskal): 간선 정렬, 사이클 없이 추가
- 프림(Prim): 정점에서 시작, 최소 간선 추가

위상 정렬

- DAG(방향 비순환 그래프)에서 정점 순서화
- 진입 차수 0인 정점부터 제거

제8장 신기술 (★★★)

8.1 인공지능 (AI/ML/DL)

AI vs ML vs DL

- AI: 인간 지능 모방하는 전체 개념
- ML: 데이터로 학습 (지도/비지도/강화)
- DL: ML의 한 분야, 신경망 다층 (CNN, RNN, Transformer)

머신러닝 학습 방법

방법	데이터	대표 알고리즘
지도	입력+정답	선형 회귀, 의사결정 트리, SVM, KNN, 분류·회귀
비지도	입력만	K-means, DBSCAN, PCA, 군집화·차원 축소
강화	환경+보상	Q-learning, DQN, 정책 경사
준지도	일부 정답	Self-training, Co-training

딥러닝 모델

- CNN: 이미지 (LeNet, VGG, ResNet)
- RNN/LSTM/GRU: 시계열, 자연어
- Transformer: 자기 어텐션, BERT, GPT
- GAN: 생성 모델 (생성자 vs 판별자)

8.2 클라우드 컴퓨팅

서비스 모델 (★★★)

모델	제공	사용자 관리	예
IaaS	인프라 (가상머신, 스토리지)	OS, 미들웨어, 앱	AWS EC2, Azure VM
PaaS	플랫폼 (OS, 런타임)	앱, 데이터	Heroku, GAE
SaaS	완성된 SW	거의 없음	Gmail, Office365

배포 모델

- 퍼블릭: 일반 사용자에게 공개 (AWS, Azure)
- 프라이빗: 단일 조직 전용
- 하이브리드: 퍼블릭+프라이빗 결합
- 커뮤니티: 특정 그룹 공유

가상화·컨테이너

구분	VM	컨테이너 (Docker)
격리 수준	게스트 OS 별도 (강함)	호스트 OS 커널 공유 (약함)
부팅 속도	분 단위	초 단위
리소스	무거움	가벼움
오케스트레이션	VMware, OpenStack	Kubernetes , Docker Swarm

8.3 블록체인

특징

- **분산 원장**: 모든 노드가 동일한 장부
- **불변성**: 해시 체인으로 변조 방지
- **합의 알고리즘**: PoW(작업증명), PoS(지분증명), PBFT
- **스마트 컨트랙트**: 조건 충족 시 자동 실행 (이더리움)

활용

- 암호화폐 (비트코인, 이더리움)
- DeFi(탈중앙 금융), NFT, DAO
- 공급망 관리, 의료 기록, 투표

8.4 IoT·5G

IoT 프로토콜

프로토콜	특징
MQTT	발행/구독 모델, 경량
CoAP	REST 기반, 경량 HTTP 대체
AMQP	엔터프라이즈 메시징
ZigBee	메시 네트워크, 저전력
BLE	저전력 블루투스
LoRaWAN	장거리 저전력
NB-IoT	셀룰러 IoT

5G 3대 시나리오

- **eMBB**: 초고속 광대역 (4K/8K, VR)
- **URLLC**: 초저지연 신뢰성 (자율주행, 원격수술)
- **mMTC**: 대규모 IoT 연결 (km²당 100만 기기)

5G 기술

- Massive MIMO (다중 안테나)
- 빔포밍 (방향성 전파)
- 네트워크 슬라이싱
- 엣지 컴퓨팅 (MEC)

8.5 빅데이터

5V

1. **Volume**: 대규모
2. **Velocity**: 빠른 생성/처리
3. **Variety**: 정형/반정형/비정형
4. **Veracity**: 정확성

5. **Value**: 가치

빅데이터 기술 스택

- **저장**: HDFS, S3
- **처리**: MapReduce, Spark
- **NoSQL**: HBase, Cassandra, MongoDB
- **분석**: Hive, Pig, Impala
- **시각화**: Tableau, Power BI

8.6 메타버스·신기술

메타버스

- ASF 분류: 증강현실, 라이프로그, 거울세계, 가상세계
- 핵심 기술: VR/AR/MR, NFT, 디지털 트윈, 아바타
- 플랫폼: 로블록스, 제페토, 디센트럴랜드

양자 컴퓨팅

- 큐비트: 0+1 중첩 상태
- 얽힘(Entanglement): 큐비트 간 상관 관계
- 쇼어 알고리즘 (RSA 깨기), 그로버 알고리즘 (검색)
- 양자 내성 암호 (PQC) 필요

DevOps·CI/CD

- **DevOps**: 개발(Dev) + 운영(Ops) 통합 문화
- **CI**: 지속적 통합 (자본 병합 + 자동 테스트)
- **CD**: 지속적 전달/배포 (자동 배포)
- 도구: Jenkins, GitLab CI, GitHub Actions, ArgoCD

MSA (마이크로서비스)

- 작은 독립 서비스로 분리, 각각 배포
- 통신: REST API, gRPC, 메시지 큐
- 단점: 분산 복잡성, 트랜잭션 어려움
- 보완: API Gateway, Service Mesh

제9장 알고리즘 (★★)

9.1 시간 복잡도

Big-O 표기

- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

주요 알고리즘 복잡도

알고리즘	평균	최악	공간
선형 탐색	$O(n)$	$O(n)$	$O(1)$
이진 탐색	$O(\log n)$	$O(\log n)$	$O(1)$
버블/선택/삽입	$O(n^2)$	$O(n^2)$	$O(1)$
퀵 정렬	$O(n \log n)$	$O(n^2)$	$O(\log n)$
병합 정렬	$O(n \log n)$	$O(n \log n)$	$O(n)$
힙 정렬	$O(n \log n)$	$O(n \log n)$	$O(1)$
기수 정렬	$O(dn)$	$O(dn)$	$O(n+k)$

9.2 알고리즘 설계 기법

분할 정복 (Divide & Conquer)

- 큰 문제를 작은 부분으로 분할 후 정복
- 예: 병합 정렬, 퀵 정렬, 이진 탐색

동적 계획법 (DP)

- 중복 부분 문제 + 최적 부분 구조
- 메모이제이션(top-down) or 타블레이션(bottom-up)
- 예: 피보나치, LCS, 0/1 배낭, LIS

그리디 (Greedy)

- 매 단계 지역 최적 선택
- 최적해 보장: MST(Kruskal·Prim), 다익스트라, 활동 선택, 분할 배낭, 허프만
- 최적해 비보장: 0/1 배낭, TSP

백트래킹

- 후보를 시도하다 실패 시 되돌아가기
- 예: N-Queen, 미로 탐색, 부분 집합 합

분기 한정 (Branch & Bound)

- 백트래킹 + 가지치기로 최적해 탐색
- 예: TSP, 0/1 배낭

제10장 멀티미디어 (★)

10.1 이미지

포맷	특징
BMP	무압축, 큰 용량
JPEG	손실 압축, 사진
PNG	무손실 압축, 투명도
GIF	256색, 애니메이션
WebP	Google, 압축 효율 ↑

색상 모델

- **RGB**: 빛의 3원색, 모니터
- **CMYK**: 인쇄용
- **HSV/HSL**: 색상-채도-명도

10.2 음성·영상

오디오

- 표본화(Sampling) → 양자화(Quantization) → 부호화(Encoding)
- 나이퀴스트: 표본화 주파수 $\geq 2 \times$ 최고 주파수
- 포맷: WAV(무압축), MP3(손실), FLAC(무손실), AAC

비디오

- 코덱: H.264(AVC), H.265(HEVC), VP9, AV1
- 컨테이너: MP4, AVI, MKV
- 압축: 키프레임(I) + 차이 프레임(P, B)

제11장 보안 (★)

11.1 암호화

대칭키

- AES (128/192/256), DES(56비트, 취약), 3DES, SEED, ARIA, IDEA
- 빠르지만 키 분배 문제

비대칭키 (공개키)

- RSA (소인수분해), ECC (타원곡선), ElGamal
- 키 분배 OK, 느림
- 디지털 서명 가능

해시

- MD5(취약), SHA-1(취약), SHA-256, SHA-3
- 단방향(원본 복원 X), 충돌 저항성

디지털 서명

- 송신자 개인키로 서명 → 수신자가 공개키로 검증
- 부인 방지, 무결성 보장

11.2 인증·인가

인증 vs 인가

- **인증 (Authentication)**: 누구인가? (로그인)
- **인가 (Authorization)**: 무엇을 할 수 있나? (권한)

OAuth 2.0

- 권한 위임 프로토콜 (인증 아님)
- Access Token으로 리소스 접근
- Grant Types: Authorization Code, Implicit, Client Credentials, Password

OpenID Connect

- OAuth 2.0 위에 인증 추가
- ID Token (JWT)

11.3 웹 보안

OWASP TOP 10 (2021)

1. 접근 통제 실패
2. 암호화 실패
3. **인젝션** (SQL, Command, LDAP)
4. 안전하지 않은 설계
5. 보안 구성 오류
6. 취약한 구성요소
7. 식별/인증 실패
8. SW·데이터 무결성 실패
9. 로깅·모니터링 실패
10. SSRF

주요 공격

- **SQL Injection**: 입력값에 SQL 삽입 → 매개변수화 쿼리로 방어
- **XSS**: 스크립트 삽입 → 출력 이스케이프
- **CSRF**: 위조 요청 → CSRF 토큰

- **DDoS**: 분산 서비스 거부 → CDN, WAF

제12장 빈출 함정 정리 (시험 직전 필수)

12.1 자주 헛갈리는 사실

함정	정답
DRAM은 비휘발성?	둘 다 휘발성 (DRAM, SRAM 모두)
SNMP는 TCP 기반?	UDP 기반 (DNS, DHCP도 UDP)
ICMP는 응용 계층?	네트워크 계층 (3계층)
RAID 6이 RAID 5보다 빠름?	RAID 6이 더 느림 (이중 패리티)
교착 4조건에 선점 포함?	‘비선점’ 포함 (선점 X)
결합도 가장 강한 것?	내용(Content) 결합도
응집도 가장 강한 것?	기능적(Functional) 응집도
후위 표기식: A+B → ?	AB+ (피연산자 먼저, 연산자 마지막)
BCNF는 3NF 만족 안 해도 됨?	3NF는 BCNF의 필요조건 (BCNF ⊂ 3NF)
Optimal 페이지 교체 구현 가능?	이론적, 미래 정보 필요 (실제 불가)

12.2 헛갈리는 포트

- **DNS = 53** (≠ DHCP=67)
- **HTTP = 80** (≠ HTTPS=443)
- **SMTP = 25** (메일 송신, POP3=110·IMAP=143는 수신)
- **SSH = 22, Telnet = 23**

12.3 정렬 알고리즘 그룹

- **O(n²) 그룹**: 버블, 선택, 삽입 (간단, 작은 데이터)
- **O(n log n) 그룹**: 퀵(최악 n²), 병합, 힙
- **O(n) 가능 그룹**: 계수, 기수, 버킷 (조건부)

12.4 자료구조 매칭

- **LIFO**: 스택 / **FIFO**: 큐
- **DFS**: 스택(or 재귀) / **BFS**: 큐
- **최소 신장 트리**: 크루스칼, 프림 (그리디)
- **최단 경로**: 다익스트라(가중치 ≥0), 벨만-포드(음수 OK), 플로이드-워셜(모든 쌍)

12.5 Java vs Python 코드 함정

```
// Java: int 나눗셈은 뿔만
System.out.println(7 / 2); // 3 (3.5 아님)

// Python: 정수/정수도 float
print(7 / 2) // 3.5
print(7 // 2) // 3 (정수 나눗셈)

// Python: 참조 vs 새 객체
y = x           # 참조
y = x + [5]     # 새 객체 (x는 영향 없음)
y.append(5)     # 같은 객체 수정 (x도 영향)
```

부록: 출제 비중 및 학습 전략

영역별 학습 우선순위

우선순위	영역	비중	학습 시간 (40h 기준)
★★★★★	컴퓨터 구조	24.4%	10시간
★★★★	운영체제	15.6%	7시간
★★★★	네트워크	13.9%	6시간
★★★	프로그래밍	10.6%	5시간
★★★	데이터베이스	7.8%	3.5시간
★★	SW공학	7.2%	3시간
★★	자료구조	6.7%	2.5시간
★	신기술	6.7%	1.5시간
★	알고리즘	3.9%	1시간
★	멀티미디어/보안	2.8%	0.5시간

시험 직전 점검 체크리스트

- ☐ 2의 보수, 진법 변환 자유롭게
- ☐ CPU 스케줄링 선점/비선점 구분
- ☐ 페이지 교체 알고리즘 3종 추적
- ☐ OSI 7계층, TCP/IP 4계층 매칭
- ☐ IPv4 클래스, 서브넷 계산
- ☐ TCP/UDP 차이, 주요 포트
- ☐ C 포인터·배열·재귀 코드 트레이스
- ☐ SQL 실행 순서, JOIN 종류
- ☐ 정규화 1NF~BCNF
- ☐ 결합도/응집도 순서
- ☐ 디자인 패턴 23종 분류
- ☐ 정렬 알고리즘 시간 복잡도
- ☐ 클라우드 IaaS/PaaS/SaaS
- ☐ bin출 함정 12.1 표 점검

시험은 6월 20일! 화이팅!